University of North Carolina at Charlotte
Department of Electrical and Computer Engineering

Homework Report 5

ECGR 5106 - Introduction to Deep Learning

Name: Viprav Lipare

Student #: 810288922

Date: July 8th, 2026

GitHub: https://github.com/vipravlipare/ECGR-5106-Intro-To-Deep-Learning

Homework 5 Report

Overall Setup and Assumptions

All experiments used PyTorch, torchvision, and CIFAR-100 loaded through torchvision.datasets.CIFAR100. CIFAR-100 has 100 image classes with 32x32 RGB images, which makes it much harder than CIFAR-10. Because the work was run on CPU hardware, the notebooks include a fast_cpu mode for practical training.

The main metrics reported are final test accuracy, best test accuracy, training loss, test loss, parameter count, FLOPs proxy, and training time. Final test accuracy is treated as the main comparison metric because it shows the final saved model performance. Best test accuracy is also reported because early stopping and learning-rate scheduling sometimes reached a stronger checkpoint before the last epoch.

FLOPs are reported as forward-pass estimates/proxies instead of exact hardware-independent counts. They are still useful because they show the relative computational cost between models. Parameter count is used as the model-size metric, and training time is reported as either average seconds per epoch or feature-extraction/head-training time depending on the experiment.

The executed notebooks and result artifacts are saved in the Homework_5 folder with visible outputs.

Problem 1: Vision Transformer from Scratch vs. ResNet-18

Objective

The goal of Problem 1 was to build a Vision Transformer from scratch for CIFAR-100 and compare several ViT configurations against a ResNet-18 baseline. The main question was whether changing patch size, embedding dimension, number of transformer blocks, and number of attention heads improved test accuracy enough to justify the extra parameters, FLOPs, and training time. The comparison uses final test accuracy as the main metric, with model size and runtime used to explain the accuracy-complexity tradeoff.

Model Setup

The scratch ViT used the standard Vision Transformer structure: patch embedding, a learnable class token, positional embeddings, transformer encoder blocks, layer normalization, and a final linear classification head for 100 CIFAR-100 classes. The patch embedding was implemented with a convolution, so each image was split into fixed-size patches and each patch became one token. The class token was used as the final image representation for classification.

Each transformer encoder block used multi-head self-attention, residual connections, layer normalization, and an MLP. The MLP hidden dimension was set to four times the embedding dimension, which follows the assignment requirement. In the full configuration idea, this means an embedding size of 256 uses an MLP dimension of 1024, and an embedding size of 512 would use an MLP dimension of 2048.

The assignment asked for experiments with patch sizes 4x4 and 8x8, embedding dimensions 256 and 512, transformer blocks 4 and 8, and attention heads 4 and 8. The notebook includes this full-assignment direction, but the saved CPU run used scaled versions so the notebook

could finish on a laptop. The reported CPU configurations still test the same design ideas: smaller vs. larger patches, lower vs. higher embedding size, shallow vs. deeper transformer blocks, and fewer vs. more heads.

ResNet-18 was used as the baseline from torchvision.models. It was adjusted for CIFAR-100 by using a 3x3 first convolution and removing the initial max pooling layer, since CIFAR-100 images are only 32x32. This made the ResNet baseline more appropriate for small images while keeping the overall residual-block design.

Training Setup and Hyperparameters

The assignment standard settings are batch size 64, 10 epochs, Adam optimizer, and learning rate 0.001. The saved CPU run kept the same batch size and starting learning rate, and used AdamW as the Adam-style optimizer because weight decay is handled more cleanly. The 10 epoch requirement was exceeded because the CPU-optimized models trained longer with early stopping so the results would be more meaningful.

The saved runs used batch size 64, initial learning rate 0.001, AdamW, weight decay 5e-4, dropout 0.20, label smoothing 0.15, ReduceLROnPlateau, gradient clipping, and early stopping. A fixed balanced CIFAR-100 subset was used in fast_cpu mode: 6,000 training images and 2,000 test images. This was done because the original full patch-size-4 ViT took about 1,700 seconds for one CPU epoch.

The same train/test preprocessing was used across the Problem 1 models so the comparison stayed fair. The practical CPU-mode results should therefore be interpreted as laptop results rather than full GPU benchmark results. This assumption is important because a full CIFAR-100 run with the larger assignment-size ViT configurations would be more appropriate on Google Colab or a local GPU.

Problem 1 Results

For the complexity comparison, parameter counts were calculated from the actual PyTorch models and checked against the theoretical layer sizes. The ViT parameter estimate includes patch embedding, class token, positional embedding, attention projections, MLP layers, layer norms, and the classification head. The FLOPs proxy estimates the relative forward-pass cost from patch projection, self-attention, MLP operations, and the final classifier.

Model	Patch	Embed	Blocks	Params	FLOPs	Sec/Epoch	Best Acc	Final Acc
ViT_fast_p8_e128_l2_h4	8	128	2	436.7k	7.24M	7.51	17.90%	21.05%
ViT_fast_p8_e256_l2_h4	8	256	2	1.66M	27.85M	27.41	17.00%	20.00%
ViT_fast_p8_e256_l4_h8	8	256	4	3.24M	54.88M	52.26	15.90%	17.90%
ViT_fast_p4_e128_l2_h4	4	128	2	424.4k	28.13M	46.82	22.15%	24.85%
ResNet18_CIFAR100	NA	NA	NA	11.22M	555.47M	241.54	35.20%	35.10%

The main comparison table includes four scratch ViT configurations and the ResNet-18 baseline. The primary metric is final test accuracy, because the assignment asks for the final model performance after training. Best test accuracy is also included to show the strongest checkpoint reached during training. Sec/Epoch measures average CPU training time per epoch, Params measures trainable model size, and FLOPs is a forward-pass complexity proxy.

ResNet-18 reached the highest final test accuracy at 35.10%, with a best test accuracy of 35.20%. The best scratch ViT was the patch-size-4, embedding-128, two-block model, which reached 24.85% final test accuracy. This means ResNet-18 generalized better on CIFAR-100, but it also required much more computation: 11.22M parameters and a 555.47M FLOPs proxy, compared with 424.4k parameters and 28.13M FLOPs for the best ViT.

The patch-size result is the most important ViT trend. The patch-size-4 model used more tokens than the patch-size-8 models, so it kept more spatial detail from each 32x32 image. That improved accuracy, but it also increased runtime to 46.82 seconds per epoch. The patch-size-8 models were much faster, especially the 128-dimensional model at 7.51 seconds per epoch, but they lost image detail and had lower accuracy.

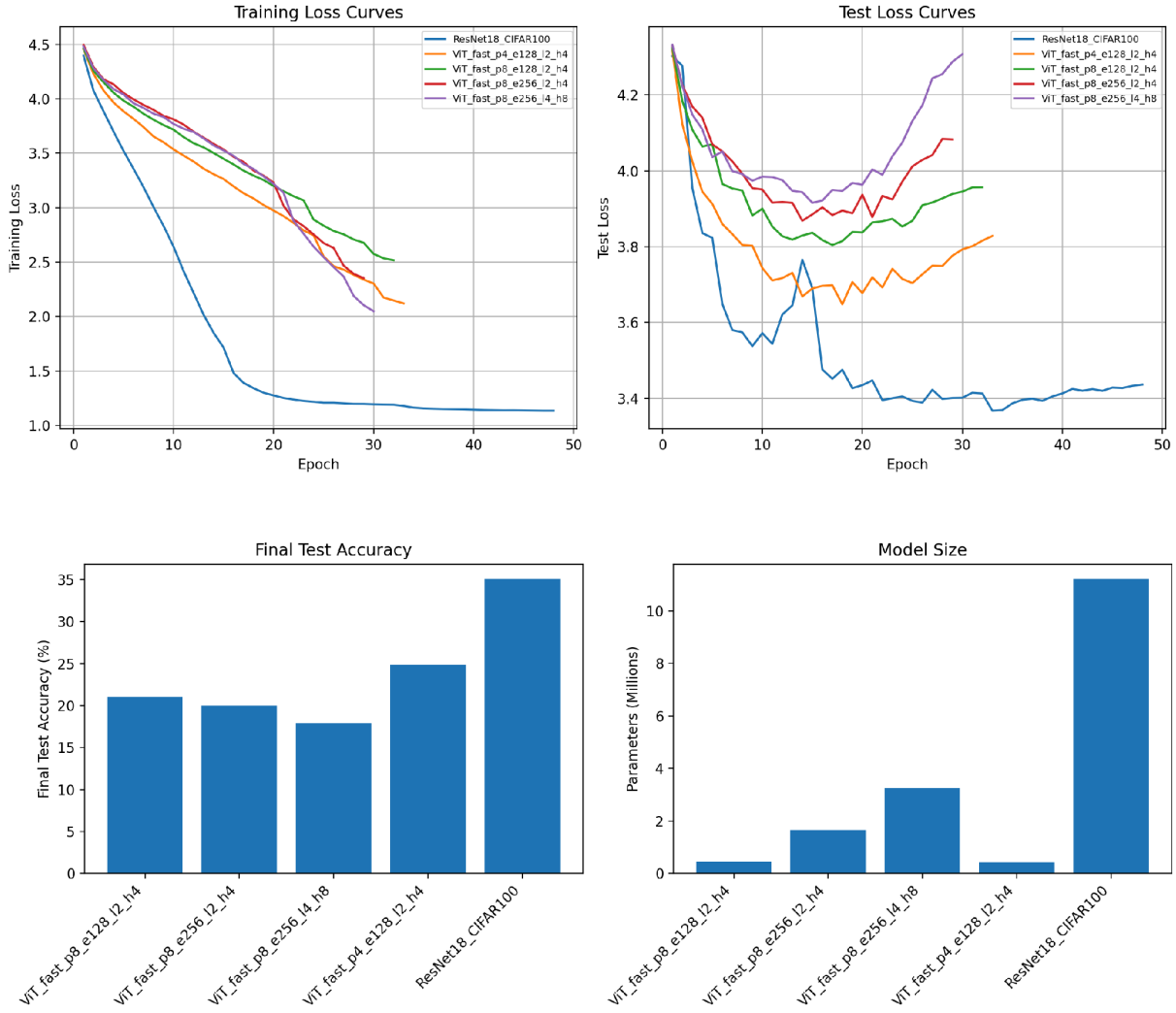
The larger ViT settings did not automatically perform better. Increasing embedding size from 128 to 256 and increasing depth/heads raised parameters and FLOPs, but test accuracy did not improve. The 4-block, 8-head ViT had 3.24M parameters and a 54.88M FLOPs proxy, but it finished at only 17.90% final test accuracy. This suggests that larger transformer capacity overfit the small CPU subset instead of learning better CIFAR-100 features.

LR-Cut Model	Best Acc	Final Acc	Epochs	LR Cuts	Final LR	Final Gap
ViT_fast_p8_e128_l2_h4_LR_Cut_Test	19.35%	20.45%	43	7	0.000133	1.479

The LR-cut test was an extra experiment on the fastest ViT model. The idea was to reduce the learning rate when the gap between training loss and test loss became large, because that gap suggested overfitting. The schedule cut the learning rate by 25% several times and ended with a final learning rate of 0.000133.

This custom schedule helped slightly, but it did not completely fix the problem. Best test accuracy improved from 17.90% to 19.35%, while final test accuracy was 20.45%. The final loss gap was still large at 1.479, so the model was still fitting the training subset better than the test subset. This result shows that learning-rate scheduling can help stability, but it cannot fully replace more data, stronger augmentation, or a better architecture.

Plot Discussion



The Problem 1 loss curves show that all models learned the training subset, but they did not all generalize equally well. Training loss kept decreasing for the ViT models and ResNet-18, while test loss flattened or started increasing after the best epoch. This is the main sign of overfitting. ResNet-18 overfit strongly because its training accuracy became very high, but its final test accuracy stayed around 35%.

The accuracy and parameter plot makes the tradeoff easier to see. ResNet-18 had the highest accuracy, but it also had the largest parameter count and the largest FLOPs proxy. ResNet-18 also took much longer to run in comparison to the other models. The scratch ViTs were smaller and cheaper, especially the patch-size-8 models, but their lower test accuracy showed that they were not as strong for CIFAR-100 under this CPU setup. The best scratch ViT was the patch-size-4 model because it used more image tokens and preserved more spatial information.

Problem 2: Fine-Tuning Pretrained Swin Transformers vs. Training from Scratch

Objective

The goal of Problem 2 was to compare pretrained Swin-Tiny, pretrained Swin-Small, and a Swin Transformer trained from random initialization on CIFAR-100. The main comparison is between fine-tuning and training from scratch. The report focuses on final test accuracy, best test accuracy, training time, parameter count, trainable parameter count, and FLOPs proxy.

Model Setup

The pretrained models were loaded from Hugging Face using SwinForImageClassification.from_pretrained(). The two checkpoints were microsoft/swin-tiny-patch4-window7-224 and microsoft/swin-small-patch4-window7-224. Both models were changed to output 100 CIFAR-100 classes by replacing the original classification head with a new 100-class head.

The backbone layers were frozen for the pretrained models, so only the new classification head was trainable. This matches the fine-tuning idea in the assignment: the model keeps the general visual features learned during pretraining, and the new head learns how to map those features to CIFAR-100 classes. Freezing the backbone also made the CPU run more practical.

The scratch Swin model was randomly initialized and trained without pretrained weights. It used a smaller Swin-style configuration in fast_cpu mode: embed_dim 32, depths [1, 1, 1, 1], heads [1, 2, 4, 8], window size 7, and a 100-class head. This model was included to show what happens when the same general Swin idea is trained from scratch instead of starting from a pretrained checkpoint.

Training Setup and Hyperparameters

The assignment setting for pretrained Swin was batch size 32, 5 epochs, Adam, and learning rate 2e-5. That setting was tested first, but with the backbone frozen, the randomly initialized classification head learned very slowly and Swin-Tiny only reached about 2% test accuracy after 5 epochs. Because the goal was to produce a useful comparison, the final reported CPU run used cached frozen-backbone features and trained the head for longer.

The final pretrained runs used 100 head-training epochs, learning rate 0.001, AdamW, label smoothing 0.05, ReduceLROnPlateau, and feature batch size 256. This is still head-only fine-tuning because the Swin backbone stayed frozen. The cached-feature approach made the run much faster because the expensive frozen backbone was only used once during feature extraction instead of being recomputed every epoch.

The scratch Swin model used a learning rate of 0.001 and trained for 15 epochs from random initialization. The same CIFAR-100 preprocessing style was used, including resizing to 224x224 for Swin models. The 224x224 input size was used because the pretrained Swin checkpoints were built around patch4-window7-224 settings, and using the original image scale avoids shape problems with the pretrained relative-position structure.

Problem 2 Results

The table uses Total Params for the full model size and Trainable for the number of weights actually updated during training. Feat Time is the one-time frozen-backbone feature extraction

time for the pretrained models, while Sec/Epoch is the average classifier-head training time per epoch after features were cached. The FLOPs column is a forward-pass proxy used to compare computational complexity.

Model	Epochs	Total Params	Trainable	FLOPs	Feat Time	Sec/Epoch	Best Acc	Final Acc
Swin-Tiny pretrained cached-head	100	27.60M	76.9k	4.35B	254.54	0.047	61.30%	59.60%
Swin-Small pretrained cached-head	100	48.91M	76.9k	8.51B	665.91	0.073	64.90%	64.60%
Swin scratch random init	15	1.25M	1.25M	178.25M	0.00	137.192	8.20%	7.80%

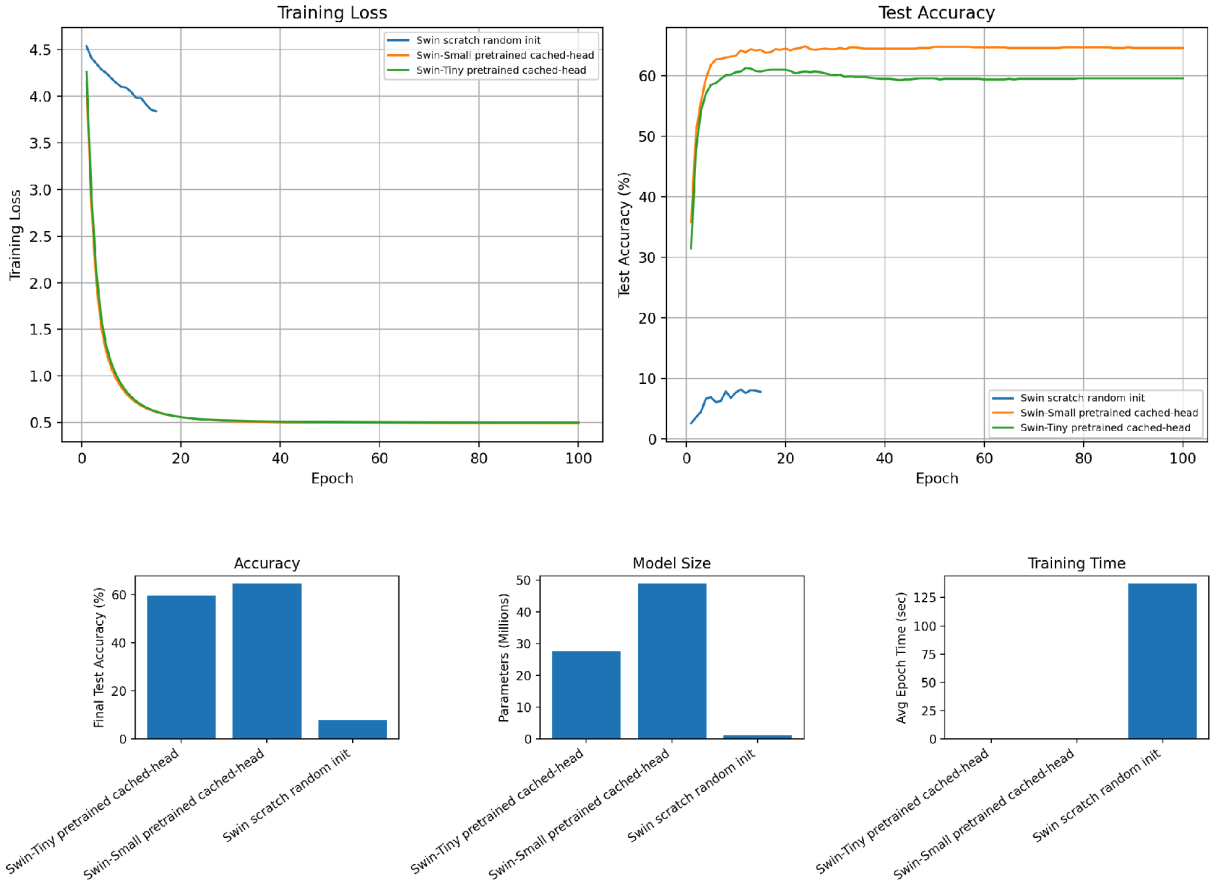
The quantitative results show that pretrained Swin fine-tuning was much better than training Swin from scratch. Swin-Small reached the highest final test accuracy at 64.60%, with a best test accuracy of 64.90%. Swin-Tiny also performed well, reaching 59.60% final test accuracy and 61.30% best test accuracy. The scratch Swin model reached only 7.80% final test accuracy, so it was far behind both pretrained models.

The table also separates total parameters from trainable parameters. This is important because the pretrained Swin models had large total parameter counts, but only 76.9k parameters were trainable after freezing the backbone. This means the final classifier head was small, but it used features from a very large pretrained model. In contrast, the scratch Swin model had 1.25M trainable parameters because all of its layers were trained from random initialization.

Swin-Small was better than Swin-Tiny because its larger pretrained backbone produced stronger image features. It had 48.91M total parameters and an 8.51B FLOPs proxy, compared with 27.60M parameters and a 4.35B FLOPs proxy for Swin-Tiny. The accuracy improvement came with a cost: Swin-Small required 665.91 seconds for feature extraction, while Swin-Tiny required 254.54 seconds.

The scratch Swin model was much cheaper in total model size and FLOPs, but it did not have pretrained representations. This explains why it underperformed. CIFAR-100 has 100 classes and only 32x32 images, so learning good hierarchical visual features from scratch is difficult in a short CPU run. Fine-tuning worked better because the pretrained models already knew useful edges, textures, object parts, and spatial patterns before seeing CIFAR-100.

Plot Discussion



The Problem 2 training curves show that the pretrained Swin models learned much faster than the scratch Swin model. After cached feature extraction, the classifier head loss dropped quickly because the inputs were already strong visual features instead of raw pixels. Swin-Tiny and Swin-Small both reached high test accuracy compared with the scratch model, while the scratch Swin stayed low even after training for 15 epochs.

The summary plot shows the main tradeoff. Swin-Small had the highest final accuracy, but it also had the most parameters and the largest FLOPs proxy. Swin-Tiny was a slightly cheaper pretrained model and still performed strongly. The scratch Swin was much smaller and had a lower FLOPs proxy, but the low accuracy shows that low complexity by itself is not enough. For CIFAR-100, pretrained feature quality mattered more than having a small randomly initialized model.

Final Comparison and Conclusion

Overall, the strongest model in Homework 5 was the pretrained Swin-Small model from Problem 2. It reached 64.60% final test accuracy and 64.90% best test accuracy, which was much higher than the best Problem 1 ResNet-18 result of 35.10% and the best scratch ViT result of 24.85%. This shows that transfer learning was the most effective strategy in this homework. Instead of learning all visual features from CIFAR-100 from scratch, Swin-Small started with pretrained ImageNet features and only needed to learn a new CIFAR-100 classification head.

Problem 1 mainly showed the tradeoff between tokenization, model size, and generalization. The patch-size-4 ViT was the best scratch ViT because it kept more image detail than patch-size 8, but it also required more computation per epoch. ResNet-18 still generalized better than all scratch ViTs, which makes sense because convolutional networks have a useful image prior for small 32x32 images. The deeper and larger ViT variants had more parameters and FLOPs, but they did not improve test accuracy, so extra capacity was not automatically useful on the smaller CPU subset.

Problem 2 showed the opposite side of the transformer story. When the transformer backbone was already pretrained, the Swin models performed very well. Swin-Small had the best accuracy, but it also had the largest backbone, highest FLOPs proxy, and longest feature-extraction time. Swin-Tiny was slightly weaker in accuracy but cheaper to run. The scratch Swin model was much smaller, but its low accuracy showed that training a hierarchical transformer from random initialization needs more data, more epochs, and more compute than the CPU setup allowed.

The main limitation is that the saved experiments were run in fast_cpu mode instead of full GPU mode. Because of that, the absolute accuracies should be interpreted as CPU-subset results rather than full CIFAR-100 benchmark numbers. However, the comparisons still answer the assignment question: ResNet-18 was stronger than the scratch ViTs under the same practical setup, pretrained Swin was much stronger than training from scratch, and accuracy improved only when the extra model complexity was paired with useful pretrained features.

Across Problems 1 and 2, the report includes the required CIFAR-100 setup, model descriptions, hyperparameters, test accuracy metrics, runtime measurements, parameter counts, FLOPs proxies, result tables, plots, and discussion of the accuracy-complexity tradeoffs. The report also states the CPU-mode assumptions so the results are interpreted honestly.

Source Files and Artifacts

Homework5_Problem1_ViT_vs_ResNet18.ipynb

Homework5_Problem2_Swin_Finetuning.ipynb

Results_Problem_1 through Results_Problem_2 folders